宸甄 PrivRAG 企业私有知识库问答系统技术文档

私有化知识库问答系统

文档版本：V1.0

编制日期：2026 年 10 月

文档性质：项目技术说明书

摘　　要

宸甄 PrivRAG 是一套面向企业的私有化知识库问答系统。系统基于检索增强生成（RAG）技术，将企业私有文档经格式感知分块与向量化后存入本地向量数据库，问答时先从私有知识库做"稠密 + 稀疏"双路语义召回，经 RRF 融合与交叉编码精排后，仅将最相关的 5 个片段送入大模型生成回答，并同步返回可点击溯源的引用来源。

系统采用前后端分离架构，前端基于 Vue 3 + Element Plus，后端基于 FastAPI + LangChain，数据层使用 MySQL 与 Qdrant 双库，支持远程 API 与本地 Ollama 双模型模式一键切换，并提供 Docker Compose 一键容器化部署。项目自建了 retrieval-only 评测框架，以 v1（80 题）/ v2（260 题）两套评测集驱动链路迭代，最终 v2 集文档级命中率 0.8115、片段级命中率 0.7846、内容召回率 0.7881，全程零幻觉。全文共 537 个单元测试用例覆盖各层。

关键词：检索增强生成；混合检索；RRF 融合；重排序；私有化部署；引用溯源；ReAct Agent

目录

第一章 项目概述 3

1.1 项目背景与业务痛点 3

1.2 项目定位与目标 3

1.3 核心能力一览 4

1.4 技术栈总览 5

第二章 需求分析 5

2.1 用户角色与权限需求 5

2.2 功能性需求 6

2.3 非功能性需求 7

第三章 系统架构设计 7

3.1 总体架构 7

3.2 后端模块结构 8

3.3 数据架构 9

3.4 部署架构 10

第四章 AI 技术方案 10

4.1 AI 能力总体蓝图 10

4.2 模型选型与选型理由 11

4.3 检索链路设计 12

4.4 AI 集成方式 13

4.5 AI 优化策略 15

4.6 评测体系与量化结果 16

4.7 技术创新点 18

4.8 应用创新点 19

第五章 核心功能实现 19
5.1 文档管理与知识入库 19
5.2 智能问答与引用溯源 20
5.3 多轮会话管理 20
5.4 RAG 参数热更新 20
5.5 账号、权限与审计 21
5.6 接口清单 21
第六章 测试与部署 21
6.1 测试体系 21
6.2 部署方案 22
6.3 环境配置 23
6.4 运维与排错要点 23
第七章 总结与展望 24
7.1 项目成果 24
7.2 已识别的局限 24
7.3 后续演进方向 25
结语 26

第一章 项目概述

1.1 项目背景与业务痛点

企业在日常运营中沉淀了大量非结构化内部文档——制度规范、产品手册、培训资料、合同模板、报表台账等。这些知识资产在传统管理模式下普遍面临三类困境：

痛点	具体表现	传统方案的局限
检索效率低下	文档分散在共享盘、OA 附件、个人电脑中，靠文件名与关键词检索	关键词匹配无法理解语义，同义表述、口语化提问几乎搜不到
通用大模型不可用	直接调用公有云大模型接口	模型未加载企业私有数据，回答易产生幻觉；且数据出域存在泄密合规风险
知识无法沉淀复用	老员工离职带走经验，新员工培训周期长	缺乏统一的知识入口与权威答案来源

与此同时，公有云大模型的普及让"私有数据不出域"成为越来越多企业的硬性合规要求，而完全本地化部署又面临算力成本与效果之间的权衡。这一矛盾正是本项目立项的出发点。

1.2 项目定位与目标

宸甄 PrivRAG 定位为**一套可私有化部署的企业知识库问答系统**，核心目标是让企业以较低成本获得"安全、准确、可溯源"的智能问答能力。项目设定四项关键目标：

1. **数据不出域**：文档原文、文本分块、向量数据全部存储在企业内网，远程模式仅将检索到的片段与问题文本传给大模型，可通过切换本地模式做到完全离线。
2. **回答可溯源**：回答与引用来源来自同一次检索结果，引用携带文档名、章节路径、页码区间与原文片段，用户可逐条核对。

3. **效果可度量**：自建评测框架，所有检索链路改动必须先跑对照实验、指标不退化才允许进入默认链路。
4. **部署可落地**：提供 Docker Compose 一键部署，基础设施与业务服务解耦，支持开发调试与生产部署两种模式。

1.3 核心能力一览

能力域	能力项	说明
知识入库	多格式文档解析	支持 pdf / docx / doc / md / xlsx / txt 六类格式，格式感知分块
知识入库	MD5 去重与向量化	同内容文档不重复入库；分块写入 Qdrant 并回写 MySQL 分块表
智能问答	混合检索问答	稠密 + 稀疏双路召回 → RRF 融合 → 精排截断 → 生成
智能问答	引用溯源	回答与引用同源，按文档去重，含章节路径与页码
智能问答	复杂问题编排	窄规则路由分流，多实体对比/多步计算走 ReAct 通道，失败自动降级
会话管理	多轮会话	会话 CRUD、按会话隔离的对话历史、消息持久化
系统治理	RAG 参数热更新	设置页保存即写入配置、同步内存并重建链路单例，无需重启
系统治理	账号与权限	管理员 + 普通用户（L1/L2/L3）两级角色，权限判定收敛到后端
系统治理	统计分析	文档总量、格式分布、入库趋势、向量化状态等看板

1.4 技术栈总览

层次	技术选型	版本/说明
前端框架	Vue 3 + Vite	组合式 API，组件化开发
前端 UI	Element Plus	企业级组件库
前端图表	ECharts	统计数据可视化
前端通信	Axios + Vue Router	统一请求封装 + 路由守卫
后端框架	FastAPI	异步高性能，自带 OpenAPI 文档
AI 编排	LangChain / LangChain Core	检索器、提示词模板、输出解析统一抽象
关系型数据库	MySQL	8.4，SQLModel + aiomysql 异步驱动
向量数据库	Qdrant	v1.16，Cosine 距离度量
本地模型服务	Ollama	本地模式下运行聊天与嵌入模型，支持自动拉取
测试框架	pytest + anyio	537 个用例，外部依赖全 mock

第二章 需求分析

2.1 用户角色与权限需求

系统采用**单租户 + 两级角色**模型，关闭自由注册，避免多租户带来的数据隔离复杂度：

角色	等级	权限范围
管理员 admin	—	文档上传/删除、知识库写操作、RAG 参数配置、账号管理（建号/改等级/启停/重置密码/软删除）、查看审计日志
普通用户 user	L1 / L2 / L3	文档查看、智能问答、个人会话

		管理、个人资料与改密；按等级开放部分知识库写权限

权限判定**唯一收敛到后端** `core/permissions.py` 的权限矩阵，由 `require_permission` 依赖统一拦截，前端仅做菜单与按钮的隐藏，杜绝"前端隐藏即可绕过"的安全隐患。管理员账号由服务启动时按环境变量 `ADMIN1_*`~`ADMIN3_*` 自动初始化（已存在则不覆盖密码），普通账号由管理员创建，账号名统一为 `czzsk` + 6 位补零自增 ID，随机初始密码**仅显示一次**。

2.2 功能性需求

编号	需求	需求描述	验收要点
F-01	用户认证	账号密码登录、JWT 鉴权、登出、修改密码	密码 bcrypt 哈希存储；连续失败锁定；`must_change_password` 强制改密
F-02	文档管理	上传、列表、预览、下载、删除	六类格式支持；MD5 去重；删除同步清理向量与分块
F-03	知识入库	文档解析 → 分块 → 向量化 → 回写分块表	入库状态可查询；失败原因落库可查
F-04	智能问答	基于私有知识库的多轮问答	检索与生成分离；回答附带引用来源
F-05	引用溯源	返回被引用的文档、章节、页码与片段	回答与引用来自同一次检索，按文档去重
F-06	会话管理	会话增删改查、消息历史	不同会话历史文件隔离；清空会话同步清理历史
F-07	参数配置	RAG 全参数读写与热更新	保存后写入配置、同步内存、重建单例并记录审计
F-08	统计分析	概览、格式分布、日趋势、向量化状态	图表与接口数据一致
F-09	账号管理	建号、改等级/状	操作全量落审计表，不含明文

		态、重置密码、软删除、审计	密码

2.3 非功能性需求

类别	需求	实现方式
数据安全	私有数据不出域	文档与向量库内网部署；支持 MODEL_MODE=local 完全离线
应答质量	抑制幻觉	仅基于检索片段作答，无命中时如实拒答；评测 Faithfulness 全程 1.0
可靠性	外部依赖降级	精排失败降回 RRF 序；ReAct 失败降级直调；链路不中断
性能	响应可控	ReAct 总超时 30s、单工具 10s；模型调用带剩余时间约束
可维护性	配置与代码分离	业务配置 YAML 化；参数热更新无需重启；配置变更留审计
可测试性	外部依赖可替换	全部模型/数据库/向量库依赖可注入、可 mock，离线跑通全量用例
可移植性	一键部署	Docker Compose 编排 6 个服务，健康检查与启动顺序控制

第三章 系统架构设计

3.1 总体架构

系统采用经典的四层前后端分离架构，各层职责边界清晰：

层次	组成	职责
接入层	Nginx	静态页面托管 + API 反向代理，统一入口
表现层	Vue 3 单页应用	登录、问答、知识库、统计、设置、账号管理六大页面

接口层	FastAPI 路由	七组路由，统一响应体 `{code, message, data}`，统一鉴权依赖
服务层	业务服务 + AI 服务	账号、文档、会话、统计、RAG 检索与生成、Agent 编排、评测
数据层	MySQL + Qdrant	关系型业务数据 + 向量数据，分块表作为两者桥梁

服务层内部进一步细分为：`service/rag`（检索与生成）、`service/agent`（ReAct 与工具）、`service/routing`（路由判定）、`service/orchestration`（混合编排）、`service/eval`（评测）、`service/history`（会话历史）、`service/observability`（trace 追踪）。

3.2 后端模块结构

模块	关键组成	说明
`modelFactory`	factory.py、rerank_factory.py	模型工厂：聊天/嵌入/精排实例化，remote/local 双模式
`service/rag/chunking`	strategy.py + 五个 chunker	按扩展名路由的格式感知分块
`service/rag/retrieval`	bm25_retriever.py、rrf.py、hybrid_retriever.py	稀疏检索、RRF 融合、链路工厂
`service/rag/query`	enhancer.py、multi_query.py、hyde.py	查询优化（可选，默认关闭）
`service/rag`	rag_service.py、registry.py、vector_store.py	检索与生成分离；单例注册与热更新；向量库读写
`service/agent`	simple_react.py、react_parser.py、tools/	极简 ReAct 循环、JSON 决策解析、工具注册
`service/routing`	rule_router.py、business_intent.py	窄规则路由 + 非业务问题识别
`service/eval`	runner.py、metrics.py、llm_judge.py 等	自建评测框架

core	permissions.py、security.py、response.py	权限矩阵、JWT 与密码哈希、统一响应

3.3 数据架构

关系型数据采用 6 张表，向量数据存放于 Qdrant 集合，二者通过**确定性 chunk ID** 建立稳定关联。

数据表	作用	关键字段
user	账号	username、password_hash、role、level、status、锁定与失败计数字段、软删除时间
document	文档	md5_hash（唯一）、file_name、file_type、vectorize_status、uploader_id
document_chunk	分块	document_id、chunk_index、content、vector_id、章节与页码元数据
chat_conversation	会话	user_id、title、时间戳
chat_message	消息	conversation_id、role、content、citations（引用来源）
audit_log	审计	actor_id、action、target_id、detail（不含明文密码）

确定性 chunk ID 设计是本项目的关键数据结构决策：以 `uuid5(命名空间, "{document_id}:{chunk_index}")` 生成 ID，同一文档同一分块在任意次重建后 ID 恒定。这带来三点收益——向量重建按 ID 覆盖写入天然幂等；RRF 融合以该 ID 为去重主键；引用溯源可稳定反查文档与章节。

3.4 部署架构

```
互联网用户 → Nginx(:18080) → Frontend Vue(:13000)  静态页面
                           → Backend API(:18000)  数据处理
                                    ↓
                        MySQL(:16666→3306)      关系型数据库
                        Qdrant(:16333→6333)     向量数据库
                        Ollama(:21434→11434)    本地模型服务（MODEL_MODE=local）
```

统一使用高位端口以规避端口占用冲突，容器内端口保持不变。所有服务配置健康检查，后端依赖 MySQL 与 Qdrant 健康后才启动。

设计点	说明
配置职责分离	`backend/app/config/` 存业务级 YAML，`docker/` 存部署级配置
多阶段构建	后端用 uv 官方镜像构建虚拟环境后复制到 slim 运行镜像；前端 Node 构建后由 Nginx 托管
运行时配置可写副本	配置目录以只读挂载作为出厂默认值，启动时复制 `retrieval_config.yml` 到运行时目录卷，解决只读挂载导致参数保存失败、且容器重建后参数不丢
环境隔离	通过 `--env-file` 切换 `.env.dev` / `.env.prod`

第四章 AI 技术方案

本章是本项目的技术核心，围绕"选什么模型、为什么这么选、怎么集成进来、如何持续优化"四个问题展开。

4.1 AI 能力总体蓝图

系统的 AI 能力由五条相互咬合的链路构成：**知识入库链路**（解析 → 分块 → 嵌入 → 落库）、**检索链路**（双路召回 → 融合 → 精排）、**生成链路**（提示

词 + 上下文 + 历史 → 回答）、**编排链路**（路由 → 直调 / ReAct → 降级）、**评测链路**（评测集 → 指标 → 报告 → 决策）。评测链路反向驱动前四条链路的每一次改动，构成闭环。

4.2 模型选型与选型理由

项目按**能力职责**而非单一模型拆分选型，每类能力独立选型、独立可替换：

能力	远程模式选型	本地模式选型	选型理由
文本嵌入	DashScope `text-embedding-v4`	Ollama `bge-m3`	远程侧为中文语义检索优化、维度与吞吐均衡；本地侧 bge-m3 是开源中文嵌入的成熟选择，支持多粒度
精排重排	DashScope `gte-rerank-v2`	`BAAI/bge-reranker-v2-m3`（CrossEncoder）	交叉编码精排对"同主题多文档"场景区分度显著优于向量相似度；v2 集实测把片段级命中率从 0.6231 提升到 0.7846
对话生成	OpenAI 协议可配置（默认 `qwen-plus`，可切换 DeepSeek 等）	Ollama `qwen2.5:7b`	采用 OpenAI 兼容协议接入而非绑定单一厂商，供应商可平滑替换；本地侧 7B 量级在消费级显卡可运行
评测裁判	DashScope `qwen-plus`（固定）	同左	与业务聊天模型解耦，保证跨阶段历史报告口径可比

关键选型论证：

(1) **为什么不直接用通用大模型的长上下文，而做 RAG？** 企业文档量级持续增长，长上下文成本与延迟不可控，且无法提供可核对的引用来源。RAG 让回答始终锚定具体片段，这是"可溯源"这一核心需求的技术前提。

(2) **为什么嵌入与精排走 DashScope、生成可自由替换？** 嵌入与精排属于检索链路的"基础设施"，更换供应商会导致历史向量全部失效、需要重建索引，因此固定供应商以保证向量空间一致性；生成模型不落库、可热替换，

因此开放给用户自主配置。

(3) **为什么保留查询优化（multi_query / HyDE）却默认关闭？** 二者已完整实现并在 v2 集做过对照实验，实测**无净收益**——该语料以制度条款为主、问题表述规范，改写后反而引入语义漂移，同时带来额外的模型调用开销。项目选择"实现但默认关闭"，作为可随时开启的实验开关保留，这正是以评测数据而非直觉做决策的体现。

4.3 检索链路设计

生产链路为「双路召回 → RRF 融合 → 精排截断」，最终仅 5 个片段进入生成：

```
用户提问
    ↓
[查询优化] mode=off（multi_query / HyDE 为可选实验开关）
    ├──────────────────────────────────┐
    ↓                                  ↓
[稠密召回]                          [稀疏召回]
Qdrant 向量检索                BM25 倒排索引（jieba 分词）
dense_topn=20                      bm25_topn=20
    └──────────────────┬──────────────┘
                       ↓
[RRF 融合] rrf_k=60，按排名融合去重，保留 topn=20
                       ↓
[精排] gte-rerank-v2 逐条打分，取 top_k=5（异常自动降级）
                       ↓
[生成] 5 个片段拼入提示词 → 大模型回答 + 引用溯源
```

链路参数与四组合语义：

环节	关键参数	默认取值
格式感知分块	chunk_size / chunk_overlap / max / min	500 / 50 / 800 / 100

稠密召回	hybrid.dense_topn	20
稀疏召回	hybrid.bm25_topn	20
RRF 融合	hybrid.rrf_k / hybrid.topn	60 / 20
精排	rerank.remote_model / rerank.top_k	gte-rerank-v2 / 5
查询优化	query_optimize.mode	off
旧链路回退	legacy_k	3

混合检索	精排	链路行为	最终条数
关	关	纯稠密检索（等价升级前旧链路）	legacy_k = 3
开	关	dense + bm25 → RRF	hybrid.topn
关	开	dense 候选 → 精排	rerank.top_k
开	开（**当前默认**）	dense + bm25 → RRF(20) → 精排	rerank.top_k = 5

四组组合通过开关自由切换，既保证新能力的收益可量化，也保证任何时刻都能一键回退到旧行为。

4.4 AI 集成方式

`modelFactory/factory.py` 以抽象工厂模式封装模型实例化，通过环境变量 `MODEL_MODE` 在 remote / local 间切换，切换成本为零代码改动：

模式	聊天模型	嵌入模型	适用场景
remote（默认）	`ChatOpenAI`（OpenAI 兼容协议）	`DashScopeTextEmbeddings`	追求效果与响应速度，数据可接受片段外传
local	`ChatOllama`（qwen2.5:7b）	`OllamaEmbeddings`（bge-m3）	合规要求严格，要求完全离线

三项工程细节值得说明：其一，本地模式启动前自动检查 Ollama 模型是否存在，缺失则自动拉取，避免首次调用长时间等待；其二，本地模型设置 `keep_alive=3600`，规避 Ollama 默认 5 分钟卸载模型导致的二次加载延迟；其三，模型加载失败只告警不中断启动，服务可起来后由管理员补配密钥。

LangChain 社区版 `DashScopeEmbeddings` 与当前 dashscope SDK 传参格式不兼容，实际调用报 `InvalidParameter`。项目直接继承 LangChain `Embeddings` 基类自主实现 `DashScopeTextEmbeddings`，完全控制请求参数：单条传字符串、批量传列表且按 `MAX_BATCH_SIZE=10` 分批，响应按 `text_index` 排序后按原顺序返回向量。这是"在框架抽象之上保留底层控制权"的典型实践。

稠密检索、BM25 检索、混合检索、精排包装、查询优化器全部继承 `BaseRetriever`，因此彼此可任意嵌套组合——`RerankRetriever(base=HybridRetriever(...))` 即完成"融合后精排"。链路由 `build_retriever()` 工厂按配置开关组装，并支持传入 `overrides` 覆盖开关组合，供评测框架做 A/B 对照而无需改动生产配置。

`rerank_factory.py` 定义统一 `rank(query, docs)` 接口，远程走 DashScope、本地走 sentence-transformers CrossEncoder（可选依赖组，未安装则自动跳过）。所有精排调用统一经 `safe_rank()` 入口：任何异常（网络、限额、依赖缺失）均捕获后返回 RRF 序的前 top_k 条，**链路不中断、用户无感**。同时修复了 langchain_community 的 `model_validator` 会无条件覆盖模型名为已下线 v1 版本的问题，构造后回写实际模型名。

在检索链路之上叠加一层**窄规则路由**：绝大多数问题仍走 RAG 直调（行为与改造前完全一致），只有命中"多实体对比 / 多步计算 / 制度冲突判断"等明确规则的问题才进入 ReAct 复杂通道。

```
用户提问 → [非业务识别] → 问候/自我介绍/使用咨询 → 直达 LLM（不检索）
              ↓ 业务问题
        [窄规则路由] router_rules.yml（强规则直接命中；弱规则累计 ≥ 阈值 2）
              ↓                                  ↓
```

```
rag_direct（默认）                react_complex
      ↓                                ↓
 RAG 直调                    ReAct：最多 3 轮工具调用 + 1 次收尾
                             工具 rag_search / compare_calc
                             总超时 30s、单工具 10s
                             失败/超时 → 自动降级 RAG 直调
```

路由规则采用**正则 + 权重**而非 LLM 分类或向量分类，理由明确：规则判定零延迟、零成本、结果可预测可审计，且避免了"用 LLM 判断要不要用 LLM"的递归开销与不确定性。规则文件外置为 `router_rules.yml`，支持启停、权重与阈值调整。

工具侧仅提供两个高度聚焦的工具：`rag_search` 复用现有混合检索链路并输出片段摘要，`compare_calc` 支持 compare / diff / sum / condition_check 四类确定性计算——把"算术"交给代码而非大模型，从根源上消除计算幻觉。

`registry.py` 以双重检查锁实现线程安全懒加载单例，聊天与 Agent 工具共用同一 RAG 服务实例。设置页保存检索参数后，系统在锁外构建新实例（构建期间读请求继续走旧实例）、成功后锁内原子替换；构建失败则保留旧实例继续服务并记录错误，保证**参数调优过程零停机**。全部变更记录 `before→after` 审计日志。

4.5 AI 优化策略

优化维度	策略	效果
分块质量	格式感知分块：按扩展名路由专用解析器，结构化格式额外做"超长再分（不句中截断）+ 小单元合并（不跨父章节）"	修复格式缺口，v1 集命中率 0.2000 → 0.9250
嵌入质量	嵌入增强：正文前拼接 `[章节路径]` 前缀一并嵌入	term 类（专有名词/缩写）命中率 0.1667 → 0.9167
召回多样性	稠密 + BM25 双路召回，jieba 搜索模式分词，仅保留正相关分参与融合	缓解纯向量的语义漂移与专名漏召

融合公平性	RRF 按排名而非分数融合，天然消除两路量纲差异，无需调权	v2 集命中率 0.6923 → 0.7462
精度收敛	交叉编码精排把 20 条候选收敛到 5 条	片段级命中率 0.6231 → 0.7846
索引一致性	BM25 索引增量维护（上传/删除事件驱动），增删后整体重算 IDF	保证 IDF 与语料一致，避免增量导致统计漂移
确定性	确定性 chunk ID + 幂等 upsert	重建索引不产生重复向量，引用可稳定反查
可靠性	精排降级、ReAct 降级、模型调用超时兜底	外部依赖故障不影响主链路可用
成本与延迟	查询优化默认关闭；ReAct 严格限流（3 轮 / 30s）	避免无收益的模型调用开销
可观测性	trace JSONL 落盘，支持 summary / full 两级	复杂链路可回放每轮 thought/action/observation

4.6 评测体系与量化结果

项目自建 retrieval-only 评测框架，指标口径明确定义：

口径	定义
文档级命中	检索结果 `document_id` 等于 ground truth
片段级命中	chunk 与溯源段落字符 2-gram 重叠率 ≥ 0.6
内容召回率	正确文档结果中最大重叠率的均值
Faithfulness	答案论断被检索上下文支持的程度（LLM 裁判 0~1）
AnswerRelevancy	答案与问题的相关性（LLM 裁判 0~1）

Ground truth 采用"document_id + 溯源原文段落"而非 vector_id，保证同一份评测集跨阶段长期可复用；评测集带 MD5 指纹写入报告头，评测集一改即失配，可即时发现口径漂移。

v1 评测集（80 题）主线结果：

指标	P0 基线	P1 格式感知分块	P2 混合检索 + 精排
文档级 / 片段级 HitRate	0.2000 / 0.2000	0.9250 / 0.9125	**1.0000 / 1.0000**
文档级 / 片段级 MRR	0.2000 / 0.1812	0.8958 / 0.8396	**1.0000 / 1.0000**
内容召回率	0.2000	0.9114	**0.9975**
Faithfulness	1.0000	1.0000	1.0000
AnswerRelevancy	0.2925	0.9338	**0.9875**

P2 在 v1 集出现双满分天花板效应，项目据此**主动迭代出更难的 v2 评测集**（260 题 / 68 份高度近似制度文档），继续做对照实验。

v2 评测集（260 题）主线结果：

指标	旧链路 k=3	稠密 k=5	混合检索	混合 + 精排（生产默认）
文档级 / 片段级 HitRate	0.6000 / 0.4615	0.6923 / 0.5115	0.7462 / 0.6231	**0.8115 / 0.7846**
文档级 / 片段级 MRR	0.4949 / 0.3551	0.5152 / 0.3662	0.5389 / 0.4512	**0.5813 / 0.5615**
内容召回率	0.4847	0.5441	0.6441	**0.7881**

P5 混合架构 A/B（10 简单 + 10 复杂）：A = 强制 RAG 直调，B = 窄规则路由混合。

指标	A 简单	B 简单	A 复杂	B 复杂
片段级 HitRate	80.00%	80.00%	90.00%	**100.00%**
答案正确率	0.8500	0.8500	0.6000	**0.6500**
AnswerRelevancy	1.0000	1.0000	0.8500	**0.9700**
平均延迟	4003ms	4995ms	4423ms	22339ms
任务完成率	—	—	70.00%	**80.00%**

结论清晰：混合架构在**简单问题上与直调完全等价**（指标一致，仅多一次规

则匹配开销），在**复杂问题上换来 +10pp 片段级命中率与 +10pp 任务完成率**，代价是平均延迟从 4.4s 升至 22.3s。两组 Faithfulness 均为 1.0000，即引入 Agent 后未产生幻觉。这是一个被量化、被权衡、且可一键关闭的架构选择。

4.7 技术创新点

序号	创新点	说明
1	格式感知分块与结构化元数据	六类格式专用解析器 + "先超长再分、后小单元合并"智能切分，分块携带章节路径与页码；嵌入时拼接章节路径增强语义，专名类召回提升 0.75
2	确定性 chunk ID 三重复用	同一 ID 同时作为 Qdrant point id、MySQL 分块外键与 RRF 融合主键，实现重建幂等、融合去重、溯源反查三位一体
3	双路召回 + RRF + 交叉编码精排的可组合链路	全部检索器实现统一抽象，四组开关组合任意切换，新能力与旧行为可同环境对照，回退零风险
4	全链路安全降级设计	精排失败降 RRF 序、ReAct 失败降直调、模型超时降级、本地模型缺失自动拉取——任一外部依赖故障均不中断主链路
5	窄规则路由的 RAG + ReAct 混合架构	以零成本、可审计的正则规则替代 LLM 分类器做分流，绝大多数请求保持直调性能，仅复杂问题付出 Agent 成本，且可一键关闭回到纯 RAG
6	检索与生成彻底分离	检索结果同时用于生成上下文与引用拼装，从架构上保证"回答所说的"与"引用所列的"必然同源
7	计算类工具化	把对比、差值、合计、条件判断交给确定性代码工具，而非让大模型做算术，从根源消除计算幻觉
8	检索参数热更新	双重检查锁单例 + 锁外构建 + 原子替换，配置调优零停机，构建失败自动保留旧实例
9	以评测数据驱动的技术决策	自建 retrieval-only 评测框架与两套带 MD5 指纹的评测集；已实现但实测无收益

		的查询优化方案被明确关闭而非盲目上线

4.8 应用创新点

序号	创新点	用户价值
1	私有化与效果的双模式平衡	remote / local 一键切换：日常用远程模型保效果，合规场景切本地完全离线，一套系统覆盖两类诉求
2	回答可溯源到页码	引用提供文档名 + 章节路径 + 页码区间 + 原文片段，用户可逐条核对，把"信不信"变成"可验证"
3	多轮会话按会话隔离	会话独立历史文件，切换会话上下文不串台；Agent 工具子查询不污染用户历史
4	复杂问题的对比计算能力	"对比两地差旅标准""判断某情形是否满足报销条件"这类需要多实体检索 + 规则判断的问题，系统可自主分步求解并给出依据
5	知识库运营可视化	文档总量、格式分布、入库趋势、向量化状态看板，让知识库从"黑盒"变成可运营的资产
6	面向企业治理的账号体系	关闭自由注册、管理员建号、初始密码仅显示一次、操作全量审计，符合企业账号管理规范
7	参数可视化调优	RAG 检索参数在设置页可视化调整并即时生效，业务人员无需改代码、无需重启即可适配自身语料

第五章 核心功能实现

5.1 文档管理与知识入库

上传流程为：接收文件 → 落盘（文件名加时间戳前缀）→ 计算 MD5 去重 → 写 `document` 记录 → 后台任务执行"解析分块 → 向量化 → 回写 `document_chunk`"。入库状态（`processing` / `success` / `failed`）与失败原因均落

库可查。删除文档时同步删除 Qdrant 向量、MySQL 分块与 BM25 索引条目，保证三处数据一致。

5.2 智能问答与引用溯源

问答主流程：`HybridOrchestrator.answer()` → 非业务识别 → 规则路由 → RAG 直调或 ReAct → 组装引用 → 写回会话历史。核心实现要点：

- **检索与生成分离**：`retrieve()` 独立执行，返回的 `docs` 同时用于拼装提示词上下文与构建引用列表，保证同源。
- **引用去重**：按 `vector_id` / `document_id` 去重，保持首次出现顺序，避免同一文档重复出现在引用区。
- **历史隔离**：`write_history` 参数控制是否写回会话历史，ReAct 内部工具调用显式传 `False`，防止子查询污染用户可见的多轮历史。
- **降级兜底**：ReAct 失败时按 `ENABLE_REACT_FALLBACK` 回落到 RAG 直调，用户端表现为一次正常回答而非报错。

5.3 多轮会话管理

会话与消息分别持久化在 `chat_conversation` / `chat_message` 表，对话历史另以文件形式按 `session_id` 独立存储，删除或清空会话时同步清理对应历史文件，避免历史残留导致的上下文串台。

5.4 RAG 参数热更新

设置页保存 → 写入 `retrieval_config.yml` → 同步内存配置 → 后台线程池调用 `registry.rebuild()` 重建链路单例 → 记录 `before→after` 审计日志。检索参数即时生效；分块参数在下次上传向量化时生效。

5.5 账号、权限与审计

权限矩阵定义在 `core/permissions.py`，是权限判定的**唯一真相源**。路由层通过 `require_permission` 依赖统一拦截，知识库写操作、RAG 参数修改、账号管理均已收紧。所有账号管理操作写入 `audit_log` 表，记录操作人、操作类型、目标账号与变更摘要，**不含明文密码**。

5.6 接口清单

路由前缀	主要接口	说明
`/api/auth`	login / logout / me / change-password	认证与个人信息
`/api/documents`	upload、list、preview、download、delete	文档全生命周期
`/api/chat`	conversations CRUD、messages、POST messages	会话管理与问答
`/api/statistics`	overview、file-type-distribution、daily-trend、vectorization-status	统计数据
`/api/settings`	GET/PUT rag-config	RAG 参数读写与热更新
`/api/admin`	users 增删改、reset-password	账号管理（仅管理员）
`/api/debug`	trace/{trace_id}	链路追踪调试（默认关闭）

第六章 测试与部署

6.1 测试体系

项目共 **537 个单元测试用例**，覆盖 44 个测试文件，全部 mock 外部依赖（MySQL / Qdrant / DashScope / LLM / Ollama），**未启动任何外部容器亦可全**

量通过。

层次	覆盖内容	代表性测试文件
工具层	路径、配置加载、MD5、日志、提示词加载、环境变量白名单	test_path_tool、test_config_handler、test_file_handler
模型层	模型工厂双模式、嵌入封装、六张表定义	test_factory、test_models
核心层	密码哈希、JWT、统一响应、权限矩阵	test_security、test_permissions、test_response
数据/依赖层	连接串、建表、鉴权依赖	test_db、test_dependencies
服务层	向量库、RAG 服务、分块、BM25、RRF、混合链路、精排、查询优化、注册表、引用	test_vector_store、test_chunking、test_rrf、test_hybrid、test_rerank、test_citations
路由层	认证、设置、账号管理	test_auth_router、test_settings_router、test_admin_users
Agent 与混合架构层	工具注册、ReAct 解析与循环、规则路由、非业务识别、trace	test_simple_react、test_rule_router、test_trace
评测层	评测集、指标、运行器、LLM 裁判	test_eval_dataset、test_eval_metrics、test_eval_runner

测试设计遵循三条原则：**外部依赖全 mock**（离线可跑、结果稳定）、**依赖可注入**（检索器、模型、记录器均可通过构造参数替换）、**样例动态生成**（如分块测试在临时目录生成样例文件，不依赖仓库内固定物料）。

6.2 部署方案

提供两种启动模式，覆盖开发调试与生产部署两类场景：

对比项	模式 A：基础设施模式	模式 B：完整容器化模式

启动服务	MySQL、Qdrant（本地模式加 Ollama）	全部 6 个服务
代码运行位置	本地	容器内
代码修改生效	即时（热重载）	需重新构建镜像
调试支持	IDE 断点调试	日志查看
适用阶段	开发调试	部署 / 演示 / 生产

一条命令即可拉起全部服务：

```
docker compose --env-file docker/env/.env.dev up -d --build
```

> 注意：每一条 `docker compose` 命令都必须带 `--env-file` 参数。本项目环境变量位于 `docker/env/` 而非项目根目录，Compose 不会自动加载，漏带会导致数据库密码等被置空、后端连不上 MySQL。

6.3 环境配置

复制模板后仅需填写 6 个必填项即可启动，其余变量均有默认值：

必填项	说明
`OPENAI_CHAT_MODEL`	业务聊天模型名
`OPENAI_API_KEY`	聊天模型 API Key
`OPENAI_BASE_URL`	聊天模型端点
`DASHSCOPE_API_KEY`	嵌入 / 精排 / 评测模型共用
`ADMIN1_USERNAME`	内置管理员登录账号
`ADMIN1_PASSWORD`	内置管理员初始密码（≥12 位）

6.4 运维与排错要点

问题	排查方向
后端容器 unhealthy	检查 `docker compose` 命令是否漏带 `--env-`

问题	排查方向
	`file`；确认 MySQL / Qdrant 已 healthy
保存 RAG 参数失败	确认运行时配置目录卷已正确挂载并可写
首次问答很慢	本地模式首次会拉取模型；检查 Ollama `keep_alive` 设置
精排未生效	查看日志是否有精排降级告警：检查 `DASHSCOPE_API_KEY` 与模型名是否被覆盖
端口冲突	修改 `.env` 中对应的高位宿主机端口映射

第七章 总结与展望

7.1 项目成果

本项目完成了一套可私有化部署的企业知识库问答系统，形成"知识入库 → 混合检索 → 精排 → 可控生成 → 引用溯源 → 评测闭环"的完整链路。核心成果可归纳为三点：

1. **效果上**：v2 评测集（260 题 / 68 份高度近似制度文档）文档级命中率 0.8115、片段级命中率 0.7846，相较改造前的旧链路（0.6000 / 0.4615）提升显著，且全程 Faithfulness 保持 1.0000，未引入幻觉。
2. **工程上**：537 个单元测试、外部依赖全 mock、链路开关化可回退、参数热更新零停机、容器化一键部署，具备实际交付与持续维护能力。
3. **方法上**：建立了"先有评测、后有优化"的工作方式——每一次链路改动都在同一评测集上做对照，指标不退化才允许进入默认链路；已实现但无收益的方案（multi_query / HyDE）被明确关闭而非默认开启。

7.2 已识别的局限

局限	说明

复杂问题延迟偏高	ReAct 通道平均延迟 22.3s，是直调的 5 倍，需进一步优化或异步化
高度近似文档仍有漏召	v2 集 0.8115 的文档级命中率意味着约两成问题未召回到正确文档
权限未下沉到检索	当前权限控制到接口与文档级，尚未做到检索时对无权限 chunk 的过滤
评测集为自建	缺乏行业公开基准对照，指标绝对值需结合语料难度理解
单租户模型	暂不支持多租户数据隔离

7.3 后续演进方向

按优先级推进，且**所有改动仍须先在现有评测框架上跑对照实验**：

方向	内容
链路精细化	差异化分块、查询分类与子问题拆解、元数据（部门/时间/类型）参与多路召回、参数自适应、精排动态阈值、生成后证据回查
知识图谱增强	轻量版实体绑定扩展召回；完整版三元组图谱支持关系查询与多跳推理；图谱结果并入候选池共同参与融合与精排
多模态检索	图纸、截图、表格、流程图经 VLM / OCR 解析后入库
权限感知检索	文档分组与行级权限过滤，检索时自动剔除无权限分块
性能与成本	Redis 缓存高频查询、模型动态路由与成本调度、ReAct 异步化与流式输出
数据与运维	增量更新流水线、版本快照、变更自动重索引、CI/CD 与备份恢复策略
反馈闭环	Badcase 归集、用户正负反馈回流评测集，让评测集随真实使用持续进化

结语

宸甄 PrivRAG 的价值不在于"用上了大模型"，而在于把大模型能力**约束在可验证、可度量、可回退的工程框架之内**：回答必须锚定检索到的片段，引用必须可回溯到具体页码，链路的每一次改动必须有评测数据支撑，任何一个外部依赖失效都必须有降级路径。在私有数据与大模型效果之间，本项目给出了一个经过量化权衡的、可落地的答案。